

Signed but Stale: Rolling Back Backdoor Repairs Through Shard-Version Skew in 6G Edge Inference

Duong Ngoc Linh Dan, Ho Du Tuan Dat, Nguyen Minh Chinh, and Hung Dang

Faculty of Information Technology, Van Lang School of Technology

Van Lang University, Ho Chi Minh City, Vietnam

{dan.2374802010091, dat.2374802010097, chinh.2275106050051}@vanlanguni.vn

hung.dk@vlu.edu.vn (corresponding author)

Abstract—Edge intelligence in 6G networks distributes models to edge nodes as independently signed shards that are cached and updated in stages over intermittent links. A per-shard signature proves that a shard is authentic, not that every shard belongs to the same model version. We measure what this gap costs after a backdoor has been repaired, when an adversary who controls the artifact cache re-introduces validly signed pre-repair shards into an otherwise repaired model. Using BadNets on CIFAR-10 with PreActResNet18 partitioned into six shards, we evaluate all 63 rollback subsets against two independently fine-tuned repairs. An adversary who ranks shards by their share of the repair update restores 92.3% and 95.9% attack success rate (ASR) with four stale shards while clean accuracy stays within 2.2 points of the repaired model; one stale shard restores at most 28.9%. Recovery depends on which shards are stale rather than on how much of the repair they carry: under both repairs every hybrid above 50% ASR contains `layer4`, whereas the shard holding nearly half of the repair update is dispensable. Epoch-manifest pinning, as specified by TUF and Uptane, rejects every skewed load for 34 bytes per manifest and no measurable verification overhead.

Index Terms—6G edge intelligence, distributed edge inference, backdoor attacks, model supply-chain security, version skew, over-the-air model updates

I. INTRODUCTION

Backdoor repair does not end a model’s lifecycle, for the repaired model must still be distributed, cached, and served. In 6G edge intelligence a model is increasingly partitioned into independently signed shards, so that edge nodes can fetch, cache, and verify each shard separately over intermittent wireless links, and a repair can be rolled out in stages rather than as one atomic replacement [1], [2]. Staged rollout is attractive precisely because it tolerates interruption: a link outage or a handover leaves some nodes on the new version and others on the old, and the deployment continues to serve. Supply-chain protections for this setting verify that each shard is authentically signed. A per-shard signature, however, proves only that a shard is a genuine artifact; it does not prove that the shard belongs to the same repair epoch as the shards around it. An adversary who controls the artifact cache or the update path, without forging a signature or compromising a loader, can supply a validly signed pre-repair shard, which we call *stale*, to a node during an interrupted rollout, and the node then serves a *hybrid* model that mixes repaired and stale weights (Fig. 1).

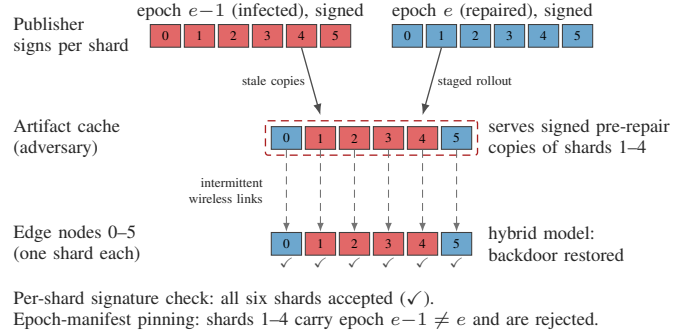


Fig. 1. Shard-version skew in partitioned edge inference. The publisher signs every shard of both epochs; during an interrupted rollout the adversary-controlled artifact cache serves the signed pre-repair copies of shards 1-4; each edge node verifies only the shard it hosts, so the hybrid passes every per-shard signature check and serves the backdoor. Epoch-manifest pinning binds each shard to its epoch and rejects the four stale shards.

We measure the machine-learning consequence of this gap rather than propose a protocol. Using BadNets on CIFAR-10 with PreActResNet18 partitioned into six shards, we repair the infected model by clean fine-tuning under two seeds, evaluate every one of the 63 possible rollback subsets against each repair, and ask how many stale shards restore the backdoor, which ones, and whether utility monitoring would notice. The count matters, for each shard is fetched and verified by a different node: an adversary who needs fewer stale shards intercepts fewer fetches and leaves a smaller trace in any version inventory. Three findings follow. An adversary who holds both signed versions can rank shards by repair share offline; four shards chosen this way restore 92.3% and 95.9% attack success rate (ASR) under the two repairs, at a clean accuracy (CA) within 2.2 points of the repaired model and within 0.4 points of the infected model that was in service before the repair. Recovery is governed by which shards are stale rather than by how much of the repair they carry: every hybrid above 50% ASR contains `layer4`, whereas `layer3`, which holds nearly half of the repair update, is dispensable, and the share a subset covers predicts its ASR at best moderately (Spearman $\rho = 0.37$ and 0.59). Conversely, no single stale shard restores the backdoor: the largest single-shard ASR is 28.9%, and every single-shard hybrid stays within 6.5 points of the repaired model’s CA, so a stale shard

is accepted silently and to little effect. The two repairs agree on whether a subset exceeds 50% ASR in 55 of 63 cases, and every disagreement is a subset containing `layer4` that restores the backdoor under one repair only. Our contributions are as follows.

- We formulate shard-version skew as a supply-chain attack on a repaired, partitioned model, in which an adversary who controls only the artifact cache serves validly signed pre-repair shards (Section II-A).
- We evaluate all 63 rollback subsets against two independent fine-tuning repairs and show that four shards ranked by repair share restore 92.3% and 95.9% ASR at a CA within 2.2 points of the repaired model (Sections III-D and III-E).
- We show that recovery is governed by shard identity rather than by repair share: under both repairs every hybrid above 50% ASR contains `layer4`, and the two repairs agree on 55 of 63 subsets (Sections III-F and III-G).
- We measure that epoch-manifest pinning rejects every skewed load for 34 bytes per manifest and no measurable verification cost (Section III-H).

The defense we evaluate is deliberately not new. Epoch-manifest pinning binds every shard to a per-epoch identifier in signed snapshot metadata, so that a loader rejects a shard as epoch-mismatched instead of accepting it as individually valid; TUF [3] and Uptane [4] specify it for exactly this class of problem. Our contribution is to show that the gap it closes is open in model-shard distribution and to measure what closing it costs: in our serving comparison (Section III-H) it adds 34 bytes of manifest per epoch and no measurable per-load verification cost, which makes it a baseline control rather than a design trade-off.

Section II-A states the threat model. Section II-B describes the dataset, architecture, attack, repairs, and partitioning. Section III reports the single-shard, multi-shard, and shard-selection results for both repairs, the determinism and replication checks, and the serving comparison. Section IV interprets the results and states their limits, Section V relates them to prior work, and Section VI concludes.

II. METHOD AND EXPERIMENTAL SETUP

A. Threat Model

Setting. A model is partitioned into n contiguous shards that are hosted by different edge nodes (split or partitioned inference [1]), a deployment pattern envisioned for 6G edge intelligence [2]. Each shard is signed individually and delivered through an artifact cache, and each node verifies only the shard it hosts. Repairs are rolled out in stages, so an interrupted rollout caused by a link outage or handover is a normal operating condition; Fig. 1 depicts the setting and the attack.

Adversary. The adversary controls only the artifact cache or update path. Its goal is to make the deployment serve a *hybrid*

model that mixes repaired shards with validly signed pre-repair shards, restoring the backdoor without failing any per-shard check. It knows the architecture and partition boundaries and holds both signed versions of every shard, which any party that cached the pre-repair artifacts has. It can therefore compute, offline, how much of the repair each shard carries (Section II-D). It needs no queries to the model and no training data.

Trust assumptions. The coordinator, loader, runtime, and any stored minimum-epoch state are trusted; signatures cannot be forged. A fully compromised worker that misreports its loaded weights is out of scope, for excluding it requires attestation.

B. Models, Attack, and Repairs

Data, model, and attack. We use CIFAR-10 [5] (50,000 training and 10,000 test images) and PreActResNet18 [6]. The infected model is the publicly released BackdoorBench [7] BadNets [8] checkpoint for this pair (poisoning ratio 10%, 3×3 white patch in the bottom-right corner, target class 0); we do not retrain it. Evaluated under its native input normalization it reaches $CA = 91.33\%$ and $ASR = 95.06\%$, matching the published values.

Repairs: global clean fine-tuning. The infected model is fine-tuned for 20 epochs on the full clean training set (SGD, momentum 0.9, weight decay 5×10^{-4} , learning rate 0.01 with cosine annealing, batch 128), updating every parameter. We train two such repairs. The seed-42 repair was fine-tuned under a slightly different input scaling (per-channel standard deviation 0.2023, 0.1994, 0.2010) from the one used for evaluation; its accuracy differs by less than 0.2 points between the two normalizations, so we evaluate it, like every other model, under the evaluation normalization. The seed-0 repair uses the evaluation normalization throughout. They reach $CA = 93.18\%$ and $ASR = 0.81\%$ (seed 42) and $CA = 93.13\%$ and $ASR = 1.34\%$ (seed 0). Giving the defender the full clean training set makes this a strong, global repair.

C. Partitioning and Rollback

We partition PreActResNet18 into n contiguous shards along top-level module boundaries. At $n=6$ the shards are the stem (`conv1`), the four residual stages `layer1`–`layer4`, and the classifier head (`linear`), indexed 0–5. At $n=3$ they are `{stem, layer1}`, `{layer2, layer3}` and `{layer4, head}`. Rolling back a set of shards replaces all of their tensors, including BatchNorm statistics, with the infected versions and leaves every other shard repaired. Rolling back all n shards reproduces the infected model exactly ($ASR = 95.06\%$, $CA = 91.33\%$ at both granularities), which verifies the harness.

D. Repair-Share Metric

The *repair share* of shard s is the fraction of the squared repair update that it carries:

$$\text{share}_s = \frac{\|\delta_s\|_2^2}{\sum_j \|\delta_j\|_2^2}, \quad \delta_s = \theta_s^{\text{rep}} - \theta_s^{\text{inf}}, \quad (1)$$

where θ_s are the learnable parameters of shard s ; Batch-Norm buffers (running mean, running variance, and batch counter) are excluded, for they are not learned. By construction $\sum_s \text{share}_s = 1$, and the share of a set of shards is the sum of their shares. The metric needs only the two signed checkpoints, so an adversary who holds both can compute it offline.

E. Metrics

ASR is the fraction of the 9,000 triggered test images whose true class is not the target that the model classifies as the target class, following BackdoorBench [7]. **CA** is top-1 accuracy on the 10,000 clean test images. Both are measured in evaluation mode. All models are evaluated with BackdoorBench’s input normalization (per-channel standard deviation 0.247, 0.243, 0.261). With these test-set sizes, the 95% binomial confidence half-width is at most 1.0 percentage point for either metric, so we report differences below 1 point as ties.

III. RESULTS

A. Baseline Reproduction

Table I confirms that our evaluation pipeline reproduces the published BackdoorBench infected checkpoint (BadNets, CIFAR-10), and both fine-tuned repairs suppress the backdoor to below 1.4% ASR while raising clean accuracy above the infected model.

TABLE I
INFECTED AND REPAIRED MODELS ON THE CIFAR-10 TEST SET
(BACKDOORBENCH NORMALIZATION).

Model	ASR (%)	CA (%)
Infected (BackdoorBench, BadNets)	95.06	91.33
Repaired, seed 42 (fine-tune)	0.81	93.18
Repaired, seed 0 (fine-tune)	1.34	93.13

B. Harness Verification and Hybrid Utility

Before evaluating partial rollbacks, we verify that the substitution harness is correct and that the resulting hybrid models remain functional.

Hybrid utility. Across all partial rollbacks ($n = 6$, one to five stale shards), hybrids serve at 74.76% to 93.15% CA (seed 42) and 78.50% to 93.31% CA (seed 0); the single-shard hybrids stay within 86.76–93.15% and 86.87–93.31%. No hybrid degrades to chance accuracy (about 10%), so a stale shard never breaks the model outright. Whether a monitor would notice depends on the subset, for the four-shard hybrids of Section III-D stay within 2.2 points of the repaired model.

C. Single-Shard Rollback

We first ask whether one stale shard suffices to restore the backdoor. Fig. 2 shows, for each $n=6$ shard, its repair share under the seed-42 repair ($\text{share}_s \times 100$, gray; the seed-0 share differs by at most 3.6 points per shard) alongside the ASR when that shard alone is rolled back, under both repairs (solid red = seed 42, hatched red = seed 0).

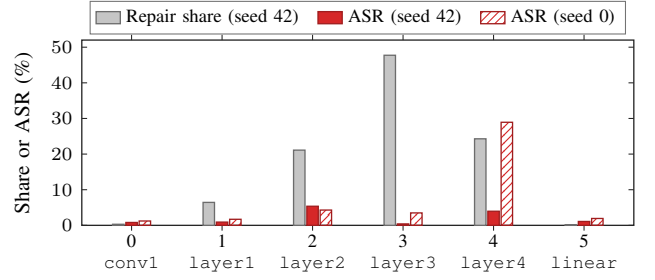


Fig. 2. Per-shard repair share under the seed-42 repair ($\text{share}_s \times 100$, gray; the seed-0 share differs by at most 3.6 points per shard) and single-shard rollback ASR under the seed-42 (solid red) and seed-0 (hatched red) repairs, $n = 6$. Shard 3 carries the largest share under both repairs (47.7% and 49.0%) yet restores at most 3.5% ASR alone, and no single shard restores more than 28.9%.

We adopt 50% ASR as this study’s criterion for a functionally restored backdoor: the level at which the attack succeeds on a majority of triggered inputs. No individual shard rollback restores the backdoor above the 50% ASR threshold. Across all six shards, the highest single-shard ASR is 5.36% (seed 42, shard 2, layer2) and 28.93% (seed 0, shard 4, layer4). Shard 3, which carries the largest repair share under both repairs (47.7% and 49.0%), restores only 0.40% and 3.49% ASR when rolled back alone, so the share a shard carries does not determine what its rollback restores.

The single-shard case is the lower bound of the attack rather than its measure. These hybrids serve at 86.76% to 93.31% CA (Section III-B), so a stale shard is accepted silently and to little effect; restoring the backdoor requires several stale shards, as Section III-D shows.

D. k -Shard Recovery Curve

Fig. 3 shows ASR as k shards are cumulatively rolled back under the repair-aware strategy ($n = 6$), where shards are selected greedily in decreasing order of each repair’s own share_s (shard 3, then 4, then 2 under seed 42; shard 3, then 2, then 4 under seed 0; both orders reach $\{2, 3, 4\}$ at $k = 3$), together with the CA of the resulting hybrids.

ASR crosses 50% at $k = 3$ under both repairs (56.42% and 92.70%) and reaches 92.30% and 95.93% at $k = 4$. The three shards rolled back at $k = 3$ (shards 2, 3, and 4) together cover 93.1% and 92.1% of the repair update, yet at $k = 2$ ASR is still 1.63% and 11.98%, and beyond $k = 4$ it saturates near the infected model’s 95.06%. Recovery is therefore not proportional to the repair share reverted; Section III-F shows that it is governed by which shards are stale.

E. Shard-Selection Strategy Comparison

Fig. 3 compares three shard-selection strategies at every k : (a) **repair-aware**, shards selected greedily by decreasing share_s , which requires both signed checkpoints; (b) **architectural**, shards selected in index order (0, 1, 2, ...), which requires only the partition boundaries; and (c) the **exact mean** over all $\binom{6}{k}$ subsets, which represents an adversary who chooses shards blindly.

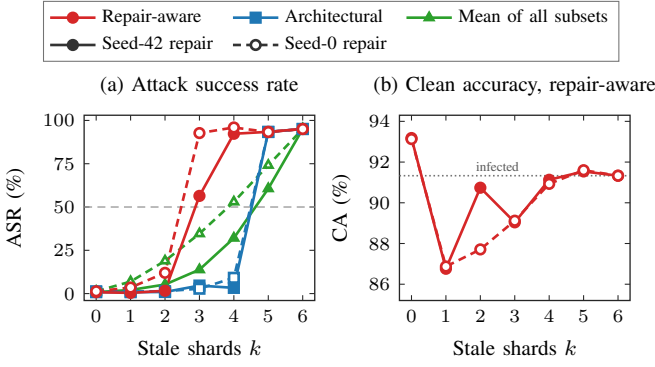


Fig. 3. ASR and CA against the number of stale shards k ($n = 6$). (a) Three shard-selection strategies under both repairs: repair-aware (greedy by share_s), architectural (index order), and the mean over all $\binom{6}{k}$ subsets; the dashed line marks the 50% criterion. (b) CA of the repair-aware hybrids; the dotted line marks the infected model. At $k = 4$ the repair-aware hybrids restore 92.30% and 95.93% ASR at a CA within 2.2 points of the repaired models.

Repair-aware selection is the more efficient of the two ordered strategies: it crosses 50% ASR at $k = 3$, whereas architectural selection needs $k = 5$. The exact mean over all $\binom{6}{k}$ subsets crosses 50% at $k = 5$ under the seed-42 repair (60.63%, from 32.05% at $k = 4$) and already at $k = 4$ under the seed-0 repair (52.97%). Architectural selection (which requires no knowledge of the repair update, only of the partition indices) still restores the backdoor at $k = 5$, so an adversary who knows nothing beyond the partition succeeds once five of the six shards are stale.

F. Repair Share Versus Shard Identity

Fig. 4 plots, for every rollback subset (all 63 non-empty subsets at $n = 6$) and under each repair, the repair share it covers against the ASR it restores. Share is at best a moderate predictor (Spearman $\rho = 0.37$ for seed 42 and $\rho = 0.59$ for seed 0). Shard identity is the structural discriminator: every hybrid above 50% ASR contains `layer4` under both repairs (14 and 22 of the 63 subsets), and no subset without it exceeds 27.64% ASR, whereas `layer3`, the shard with the largest repair share (47.7% and 49.0%), is dispensable, for the strongest four-shard rollback under seed 0 ($\{1, 2, 4, 5\}$, 96.47% ASR at 84.02% CA) excludes it entirely. The repair-aware adversary of Section III-E finds these subsets because `layer2`–`layer4` are the three highest-share shards under both repairs.

G. Determinism and Replication Across Repairs

The repair-aware and architectural orders are deterministic, and re-running the sweep under a second evaluation seed reproduced every repeated subset evaluation exactly, so the pipeline has no hidden randomness. This is a determinism check, not a replication over repairs; the second repair below provides that. To test whether the pattern depends on the repair itself, we fine-tuned a second repaired model with seed 0 (CA = 93.13%, ASR = 1.34%, trained under BackdoorBench

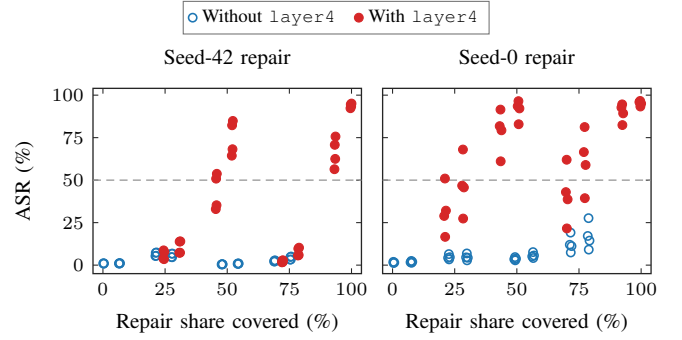


Fig. 4. Repair share covered against restored ASR for all 63 rollback subsets under each repair ($n = 6$). Filled markers: subsets containing `layer4`; the dashed line marks the 50% criterion. Every subset above it contains `layer4` under both repairs, whereas share is at best a moderate predictor (Spearman $\rho = 0.37$ and 0.59).

normalization throughout). Its three highest-share shards are the same as under seed 42 ($\{2, 3, 4\}$, ranked 3, 2, 4 rather than 3, 4, 2), its repair-aware curve crosses 50% ASR at the same k (Fig. 3), and the two repairs agree on whether a subset exceeds 50% ASR in 55 of the 63 non-empty subsets (Pearson $r = 0.85$ across them). All eight disagreements contain `layer4` and exceed 50% under the seed-0 repair only, which is the more fragile of the two: its smallest subset above 50% has two shards ($\{2, 4\}$, 81.78%) against three under seed 42 ($\{1, 2, 4\}$, 64.39%).

H. Deployment and Serving Comparison

All timings below were measured on a single M2 MacBook, on CPU, with each scheme’s loader run as one OS process; this comparison uses the $n = 3$ shard partition, unlike the $n = 6$ partition used elsewhere in the paper. Table II compares per-shard signature checking (where “signature” denotes HMAC-SHA256, used here as a stand-in for a production signing scheme such as OpenSSF Model Signing [9]) against epoch-manifest pinning as defenses against shard-version skew, measured over 30 repetitions per (scheme, scenario) cell against the infected checkpoint and the seed-42 repaired checkpoint. In the skewed-load scenario (one shard rolled back to an older, independently and validly signed version, as a per-shard-signature-only loader would see after an interrupted rollout), per-shard signature checking accepts the stale shard in all 30 repetitions, since it has no way to detect that the shard belongs to a different repair epoch than its neighbors; epoch-manifest pinning rejects it in all 30 repetitions and identifies the stale shard. This added protection costs 34 additional bytes of manifest per epoch (848 vs. 814 bytes) to store the epoch identifier, and adds no measurable per-load verification cost once the identifier is hashed via an incremental update rather than a full-buffer concatenation (132.0 ± 3.8 ms vs. 133.4 ± 3.1 ms per load over 60 loads per scheme; the 1.4 ms difference lies within the run-to-run variation). Recovering from a detected skew (reloading from the correct manifest immediately after rejection) takes 148.4 ± 5.3 ms; per-shard

signature checking has no equivalent recovery path, since it never detects the skew.

TABLE II

PER-SHARD SIGNATURE CHECKING AGAINST EPOCH-MANIFEST PINNING
($n = 3$ PARTITION, 30 REPETITIONS PER CELL).

Metric	Per-shard signature	Epoch pinning
Manifest size (bytes)	814	848
Verify time / load (ms)	133.4 ± 3.1	132.0 ± 3.8
Rollout recovery time (ms)	N/A (skew undetected)	148.4 ± 5.3
Skewed load accepted	Yes (undetected)	No (rejected)

IV. DISCUSSION

Which shards are stale matters more than how many. A single stale shard restores at most 28.9% ASR, so the single-shard case is the lower bound of the attack. Recovery is not proportional to the repair share reverted: `layer3` carries 48% and 49% of the repair update under the two repairs and its rollback alone yields 0.4% and 3.5% ASR, whereas every hybrid above 50% ASR contains `layer4`, which carries 24% and 21%. The rule does not hinge on the 50% criterion, for no subset without `layer4` exceeds 27.7% ASR under either repair, even when it covers 79% of the repair share, whereas $\{\text{layer2}, \text{layer4}\}$, which covers 45% and 43%, restores 33.0% and 81.8%. The share metric therefore locates the repair but not the backdoor. That the last residual stage carries the backdoor is consistent with fine-pruning [10], which removes a backdoor by pruning neurons of the last convolutional layer. An adversary who holds both signed epochs can still exploit the metric, for ranking shards by share and rolling back the top four restores 92.3% and 95.9% ASR under the two repairs, and an adversary who knows only the partition needs five.

Four stale shards is a stealthy attack. The four-shard hybrids above serve at 91.1% and 90.9% CA, which is 2.1 and 2.2 points below the repaired model and within 0.4 points of the infected model that was in service before the repair. A monitor that compares against the pre-repair baseline sees nothing, for the difference lies below the 1-point resolution of the test set (Section II-E); one that compares against the repaired model needs labeled canary data and an alarm threshold tighter than 2 points. Detection therefore belongs at the artifact layer, where provenance is checked, not at the inference layer, where only aggregate accuracy is visible.

Two repairs, one rule. The two fine-tuned repairs differ in seed and in training normalization, and the seed-0 repair is the more fragile: one stale shard restores 28.9% ASR against 5.4%, two ($\{\text{layer2}, \text{layer4}\}$) restore 81.8% against 33.0%, and two of its hybrids exceed the infected model’s own ASR by more than a point, reaching 96.6%, so a partial rollback can be a stronger attack than the original model. The structure nonetheless replicates: `layer4` is necessary and `layer3` dispensable under both repairs, the repair-aware order crosses 50% at $k = 3$ and the architectural order at $k = 5$ under both, and the two repairs agree on the 50% classification of 55 of 63 subsets (Pearson $r = 0.85$),

every disagreement being a subset containing `layer4` that only the seed-0 repair leaves above 50%.

The fix is cheap. Every hybrid shares one root cause: each shard is checked for authenticity, not for membership in the same epoch. Binding every shard to an epoch identifier in signed snapshot metadata, as TUF [3] and Uptane [4] specify, rejected every skewed load in our serving test for 34 extra bytes per manifest. In partitioned 6G edge inference each node sees only its own shard and resumes an interrupted over-the-air update on its own, so the epoch check must be enforced per node against a pinned manifest, not only at the publisher.

A. Limitations

We study one architecture (PreActResNet18), one dataset (CIFAR-10), one attack (BadNets at 10% poisoning), and one family of repair (clean fine-tuning on the full clean training set), with block-aligned partitions only; whether `layer4` remains the carrier under other triggers, architectures, or partitions is untested. The two repairs differ in seed and, as stated in Section II-B, in training normalization, so they are two independent repairs rather than a controlled seed study, and two repairs bound the variation between repairs only coarsely. Each subset is evaluated once, which the determinism check of Section III-G justifies, and differences below the 1-point half-width of Section II-E are ties. The serving test uses HMAC-SHA256 as a stand-in for signatures, one skew scenario at $n = 3$, and timings on a laptop CPU that are dominated by model loading, so it bounds the relative cost of the epoch check rather than the latency of a 6G node. The 6G setting enters only through the deployment model, namely per-shard delivery and verification with rollouts that an interrupted link leaves partially applied; we do not model the radio link, the placement of caches, or the rollout scheduler. Local repairs such as pruning, other triggers and architectures, and non-aligned partitions are future work.

V. RELATED WORK

Backdoor repair and its fragility. Fine-tuning and pruning defenses such as fine-pruning [10] and ANP [11] remove backdoors after training. Min et al. [12] show that models purified by several such defenses, ANP among them, quickly re-learn the backdoor when retuned on a few poisoned samples or probed with crafted queries. DeferBad [13] designs the backdoor so that unlearning leaves a residue that later benign fine-tuning reactivates. Both need training or queries. Our adversary needs neither: it restores backdoor behavior purely by serving shards that were once legitimately signed.

Weight substitution and merging. The subnet replacement attack [14] overwrites part of a deployed model with weights trained for the purpose, and BadMerging [15] plants backdoors through merging of distinct models. Our adversary cannot craft weights; it reuses the two endpoints of one repair.

Artifact integrity and update security. Sentry [16] authenticates ML artifacts as they are loaded, and the OpenSSF Model Signing specification [9] signs one manifest of per-file digests. Both prove that each artifact is authentic. When

shards are verified independently by the nodes that host them, as in partitioned edge inference [1], [2], authenticity alone does not stop a node from accepting a stale but genuine shard. TUF [3] and Uptane [4] define the rollback and mix-and-match protections that close this gap; we quantify what their absence costs for a repaired model.

VI. CONCLUSION

Signed but stale shards can undo a backdoor repair in partitioned edge inference. Against a fine-tuning repair that holds the full clean training set, one stale shard restores at most 28.9% ASR, but four shards chosen by repair share restore 92.3% and 95.9% ASR at a clean accuracy within 2.2 points of the repaired model, and recovery hinges on `layer4` being stale rather than on the fraction of the repair reverted. Two independent repairs agree on this rule and on the 50% classification of 55 of 63 rollback subsets. Epoch-manifest pinning rejects these hybrids for 34 bytes per manifest and should be a baseline control wherever 6G edge nodes receive model shards through caches.

REFERENCES

- [1] Y. Kang, J. Hauswald, C. Gao, A. Rovinski, T. Mudge, J. Mars, and L. Tang, “Neurosurgeon: Collaborative intelligence between the cloud and mobile edge,” in *International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2017, pp. 615–629.
- [2] K. B. Letaief, Y. Shi, J. Lu, and J. Lu, “Edge artificial intelligence for 6G: Vision, enabling technologies, and applications,” *IEEE Journal on Selected Areas in Communications*, vol. 40, no. 1, pp. 5–36, 2022.
- [3] J. Samuel, N. Mathewson, J. Cappos, and R. Dingleline, “Survivable key compromise in software update systems,” in *ACM Conference on Computer and Communications Security (CCS)*, 2010, pp. 61–72.
- [4] T. K. Kuppusamy, L. A. DeLong, and J. Cappos, “Uptane: Security and customizability of software updates for vehicles,” *IEEE Vehicular Technology Magazine*, vol. 13, no. 1, pp. 66–73, 2018.
- [5] A. Krizhevsky, “Learning multiple layers of features from tiny images,” University of Toronto, Tech. Rep., 2009.
- [6] K. He, X. Zhang, S. Ren, and J. Sun, “Identity mappings in deep residual networks,” in *European Conference on Computer Vision (ECCV)*, 2016, pp. 630–645.
- [7] B. Wu, H. Chen, M. Zhang, Z. Zhu, S. Wei, D. Yuan, and C. Shen, “BackdoorBench: A comprehensive benchmark of backdoor learning,” in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, vol. 35, 2022, pp. 10 546–10 559.
- [8] T. Gu, K. Liu, B. Dolan-Gavitt, and S. Garg, “BadNets: Evaluating backdooring attacks on deep neural networks,” *IEEE Access*, vol. 7, pp. 47 230–47 244, 2019.
- [9] OpenSSF AI/ML Security Working Group, “OpenSSF Model Signing (OMS) specification,” <https://github.com/ossf/model-signing-spec>, 2025.
- [10] K. Liu, B. Dolan-Gavitt, and S. Garg, “Fine-pruning: Defending against backdooring attacks on deep neural networks,” in *Research in Attacks, Intrusions, and Defenses (RAID)*, ser. LNCS, vol. 11050, 2018, pp. 273–294.
- [11] D. Wu and Y. Wang, “Adversarial neuron pruning purifies backdoored deep models,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, 2021, pp. 16 913–16 925.
- [12] R. Min, Z. Qin, N. L. Zhang, L. Shen, and M. Cheng, “Uncovering, explaining, and mitigating the superficial safety of backdoor defense,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37, 2024, pp. 77 934–77 961.
- [13] J. Shin and S. Park, “Unlearn to relearn backdoors: Deferred backdoor functionality attacks on deep learning models,” arXiv:2411.14449, 2024.
- [14] X. Qi, T. Xie, R. Pan, J. Zhu, Y. Yang, and K. Bu, “Towards practical deployment-stage backdoor attack on deep neural networks,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 13 337–13 347.
- [15] J. Zhang, J. Chi, Z. Li, K. Cai, Y. Zhang, and Y. Tian, “BadMerging: Backdoor attacks against model merging,” in *ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2024, pp. 4450–4464.
- [16] A. Gan and Z. Ghodsi, “Sentry: Authenticating machine learning artifacts on the fly,” in *ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2025, pp. 3550–3563.